



Map Prop Engine – Quick Reference

This is a lightweight HTML/JS map engine built for **digital props on film shoots**.

It behaves like a simple Google-Maps viewer: panning, smooth zooming, multiple image levels (LODs), grids, POIs, and togglable overlay layers.

Folder Structure

```
maps/data/
├── 3/
"scene"
├── 11/
└── 67/
    ├── config.json
presets, etc
    ├── points.json
    └── assets/
        ├── sat_low.png
        ├── sat_mid.png
        ├── sat_hi.png
        ├── roads_low.svg
        ├── roads_mid.svg
        └── roads_hi.svg
```

→ scene specific data, loaded by URL param

→ configuration: world size, LODs, zoom,

→ list of points of interest (POIs)

→ map data for this scene

config.json Structure

Defines map geometry, zoom limits, and image layers per zoom level (LOD).

```
{
  "worldWidth": 10000,
  "worldHeight": 10000,
  "initialView": { "x": 2048, "y": 2048, "z": 0.25 },
  "marker": { "display": true, "x": 2200, "y": 1800, "angle": 45 },
  "minZoom": 0.05,
  "maxZoom": 4,
  "forceType": "Mallorca",
  "scaleAtZoom1": 1.5, // = real-world units per pixel at z=1 (e.g.,
metersPerPixelAtZ1)
  "lods": [
    {
      "maxZ": 0.3,
      "grid": { "size": 200, "color": "rgba(255,0,0,.5)" },
      "layers": [
        { "url": "sat_low.png", "type": "base", "width": 2028,
```

```

"height": 1488, "offsetX": 0, "offsetY": 0 },
  { "url": "sat_low.png", "type": "base", "width": 2028,
"height": 1488, "offsetX": 2028, "offsetY": 0 },
  { "url": "roads_low.svg", "type": "overlay", "width": 1600,
"height": 900, "offsetX": 0, "offsetY": 0 }
]
},
{
  "maxZ": 0.9,
  "grid": { "size": 100, "color": "rgba(0,255,0,.5)" },
  "layers": [
    { "url": "sat_mid.png", "type": "base", "width": 2448,
"height": 1530, "offsetX": 0, "offsetY": 0 },
    { "url": "roads_mid.svg", "type": "overlay", "width": 1600,
"height": 900, "offsetX": 500, "offsetY": 0 }
  ]
},
{
  "maxZ": 99,
  "grid": { "size": 50, "color": "rgba(0,128,255,.5)" },
  "layers": [
    { "url": "sat_hi.png", "type": "base", "width": 2640,
"height": 1588, "offsetX": 0, "offsetY": 0 },
    { "url": "roads_hi.svg", "type": "overlay", "width": 1600,
"height": 900, "offsetX": 0, "offsetY": 0 }
  ]
}
],
"presets": [
  { "label": "HQ Gate", "x": 1800, "y": 1800, "z": 0.5 },
  { "label": "Warehouse", "x": 3100, "y": 2100, "z": 0.8 },
  { "label": "Main Sq.", "x": 2048, "y": 2048, "z": 1.2 }
]
}

```

scaling:

width height needs the same proportions as the imgs, but no necessarily be the dimensions of the img.
so you can scale with it\$

use the bolt-button and the debug button to debug the position of the lods by using the dev tools of
your browser to adjust for left/right and fill those as offsetsX/Y.

◆ Fields

Key	Description
<code>worldWidth, worldHeight</code>	total world pixel size (sets coordinate system)
<code>minZoom, maxZoom</code>	zoom limits
<code>initialView</code>	starting camera position

Key	Description
<code>initialMarker</code>	marker position (stays constant size)
<code>scaleAtZoom1</code>	meters per pixel at zoom = 1 (for scale bar)
<code>presets</code>	optional named camera positions (fly-to)
<code>lods</code>	list of image sets per zoom range
<code>grid</code>	optional debug grid for that LOD
<code>layers</code>	individual image/SVG files to draw

Each `layer` has:

- `url`: path to image or SVG
- `type`: category (used for toggling on/off)
- `width` / `height`: native pixel dimensions
- `offsetX` / `offsetY`: optional placement offset

`points.json` (POIs)

Defines interactive points of interest (icons, text, tooltips, actions).

```
[
  {
    "tooltip": "This is videocam 1234123-A",
    "icon": "videocam",
    "x": 666,
    "y": 333,
    "minZ": 0.4,
    "maxZ": 2.5,
    "size": 2,
    "iconClasses": "",
    "iconStyles": "",
    "text": "Lorem Ipsum",
    "action": ""
  },
  {
    "tooltip": "This is videocam 1234123-B",
    "icon": "location_pin",
    "x": 999,
    "y": 333,
    "minZ": 0,
    "maxZ": 4,
    "size": 2,
    "pointClasses": "red text-shadow--white noBox",
    "iconStyles": "",
    "text": "",
    "action": "alert('cam A - AAAAAA')"
```

```
}  
]
```

Hint: Add class **noBox** to a poi so there's no box around it. The bottom center of the icon is then at the defined point.

Key	Meaning
x, y	world coordinates
icon	emoji or short text
size	relative size (1 = 10 px base)
minZ / maxZ	visible zoom range
iconClasses / iconStyles	optional CSS styling
action	JS snippet to execute on click



UI Controls

Control	Action
Drag	Pan
Scroll / + / -	Zoom about cursor
	Move view
Center	Return to initial view
1-9	Jump to preset views
Layers buttons	Toggle layer types (base , overlay , etc.)



Adding New Imagery

1. Add your new file to **assets/**.
2. Add it to a layer entry in **config.json** under the appropriate LOD.
3. Give it a unique **"type"** if you want it toggleable separately.
4. Reload the map — all assets are preloaded automatically.



Grids

Each LOD can define its own grid overlay:

```
"grid": { "size": 100, "color": "rgba(255,255,255,0.2)" }
```

-
- **Size:** grid cell spacing in world pixels
 - **Color:** any CSS color
 - Grid lines zoom **with the map**, but their thickness stays constant on screen.
-

Notes

- All coordinates are in world pixels.
 - No external dependencies — everything runs in a single HTML file.
 - Ideal for playback on Raspberry Pi / Caddy servers for on-set props.
 - Images and SVGs can be any aspect ratio.
-